

-- { БАЗОВЫЙ УРОВЕНЬ - ОСНОВЫ PYTHON } --

Ethical Hacking Tutorial Group The Codeby

представляет



PYTHON

«Для Пентестера»

Programming Language..



-- { ПРОДВИНУТЫЙ УРОВЕНЬ } --

Модуль re поддерживает следующие флаги (в скобках короткий вариант флага):

- re.ASCII (re.A)
- re.IGNORECASE (re.I)
- re.MULTILINE (re.M)
- re.DOTALL (re.S)
- re.VERBOSE (re.X)
- re.LOCALE (re.L)
- re.DEBUG

Флаг re.IGNORECASE (re.I)

Игнорирует регистр.

```
import re

text = 'Адрес: Москва, Ямская 10-16'
res = re.compile('москва', re.I)
result = res.search(text)
print(result.group())
```

regex x

Москва

Флаг re.DOTALL (re.S)

Сделает соответствие метасимвола 'точка' любому символу, включая символ новой строки. В данном примере без флага мы получили бы только символы ДО знака переноса строки.

```
import re

text = 'Ваня\nМаша'
res = re.compile('.*', re.S)
result = res.search(text)
print(result.group())
```

regex x

Ваня

Маша

Флаг re.DEBUG

Показывает отладочную информацию о скомпилированном выражении.

```
import re

text = 'Ваня\nМаша'
res = re.compile('.*', re.DEBUG)
result = res.search(text)
print(result.group())
```

regex x

```
MAX_REPEAT 0 MAXREPEAT
ANY None

0. INFO 4 0b0 0 MAXREPEAT (to 5)
5: REPEAT_ONE 5 0 MAXREPEAT (to 11)
9. ANY
10. SUCCESS
11: SUCCESS
Ваня
```

Флаги можно комбинировать через плюс.

```
import re

text = 'Ваня\nМаша'
res = re.compile('^[a-я].*', re.S + re.I)
result = res.search(text)
print(result.group())
```

regex x

```
Ваня
Маша
```

Флаг **re.VERBOSE (re.X)**

Позволяет оставлять комментарии к паттернам регулярных выражений.

```
import re

text = 'Сейчас ремя в Лондоне 16:51'
res = re.compile(r'\d+:\d+ # поиск времени часы/минуты', re.X)
result = res.search(text)
print(result.group())
```

regex x

```
16:51
```

Флаг `re.ASCII (re.A)`

Флаг указывает Python сопоставлять `\w`, `\W`, `\b`, `\B`, `\d`, `\D`, `\s`, `\S` символам ASCII, вместо использования Юникода. В данном примере без флага вывелась бы вся строка.

```
import re

text = 'Сейчас время в Лондоне 16:51'
res = re.compile(r'\w+', re.A)
result = res.search(text)
print(result.group())
```

regex x

16:51

```
import re

text = 'Что это за 3 иероглифа 訓読み'
res = re.compile(r'\w+', re.A)
result = res.search(text)
print(result.group())
```

regex x

3 иероглифа 訓読み

Флаг `re.MULTILINE (re.M)`

Метасимволы символы `^` и `$` соответствуют началу и концу ЛЮБОЙ строки. Применяется в многострочном тексте.

```
import re

text = '''
Никогда не теряй,
Не теряй своей мечты
Твердо верь, твердо знай:
Всё на свете можешь ты
'''

result = re.search(r'^Всё.*', text, re.MULTILINE)
print(result.group())
```

regex x

Всё на свете можешь ты

Флаг `re.LOCALE (re.L)`

Этот флаг не имеет смысла использовать, он устарел и работает только с 8-битными локалями. В документации прямо об этом сообщается. Идея была чтобы для `\w`, `\W`, `\b`, `\B` выполнять сопоставление без учета регистра в зависимости от вашей текущей локали.

Составление паттернов

Ищем емейлы.

```
import re

string = 'Мой новый емейл pupkin@mail.ru, а старый Dodik_100@gmail.com.'
res = re.findall(r'([\w_]+@[ \w.]+\w{2,3})', string)
print('\n'.join(res))
```

```
reg x
pupkin@mail.ru
Dodik_100@gmail.com
```

Все паттерны составляются слева-направо, и так как по сути мы делаем конструктор, то любые более сложные выражения всегда создаются частями.

В первой части емейла почтовые сервисы часто допускают только буквы и цифры, а также символ нижнего подчёркивания. Поэтому мы написали `[\w_]` то есть это любой символ буквы или цифры, а также нижнее подчёркивание. Плюс находит одно и более вхождений. Далее во второй части идёт символ собачки, опять любой символ и точка `[\w.]` также + для поиска вхождений.

Так как доменные имена в большинстве случаев имеют 2 или 3 символа то мы установили от 2-х до 3-х вхождений `{2,3}`. Встречаются варианты и с более длинными доменными именами, поэтому манипулируя длиной, мы можем отлавливать доменные имена в нужном диапазоне.

Это пожалуй самая простая модель паттерна. На самом деле нет единого стандарта которому следуют во всём мире. Например GMAIL и Яндекс разрешает ставить плюс в адресе, кроме этого весьма часто разрешается в имени пользователя использовать и точку. Учтём эти моменты и составим другой паттерн:

```
import re

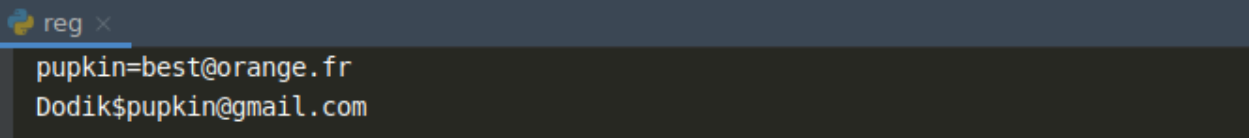
string = 'Мой новый емейл pupkin+friends@yandex.ru, а старый Dodik.pupkin@gmail.com.'
res = re.findall(r'([\w_+.]+@[ \w.]+\w{2,3})', string)
print('\n'.join(res))
```

```
reg x
pupkin+friends@yandex.ru
Dodik.pupkin@gmail.com
```

В [википедии](#) есть информация в которой указаны разрешённые символы в логине `!#$%&'*+/-=?^_`{}~` то есть к сожалению мы не можем сделать абсолютно универсальный паттерн, поскольку требования к имени логина и доменного имени весьма разные. Поэтому если хочется сделать максимально универсальный (жадный) вариант, то можно написать его следующим образом:

```
import re

string = 'Мой новый емейл pupkin=best@orange.fr, а старый Dodik$pupkin@gmail.com.'
res = re.findall(r'(\S+@\S+\w{2,3})', string)
print('\n'.join(res))
```



Однако следует понимать что такой паттерн захватит абсолютно всё - будут пойманы в тексте и любые выражения, имеющие знак @, при этом не являющиеся адресами электронной почты.

Ищем телефоны.

С телефонами тоже весьма широкий спектр написания – домашние, мобильные, с кодами городов и без, с дефисами или без и т.п. Поэтому конечно универсальный вариант также не получится. Обычно пишется регулярка под конкретный сайт или справочник, документ, базу и т.д. То есть в таких случаях по максимуму будет унифицировано написание телефонов.

Под домашние телефоны регулярка совсем простая типа `r'(\d{3}-\d{2}-\d{2})'` Такой вариант будет искать телефоны вида 177-66-56.

Гораздо сложнее с мобильными, также в эту категорию попадут и домашние с 3-х значным кодом города.

Напишем вариант, который будет искать сразу разные варианты написания номеров. Общие детали – начинаются номера или с 8 или +7, затем трёхзначное число, снова трёхзначное и два двузначных.

Чтобы + не был засчитан за метасимвол, нужно его экранировать обратным слэшем, а вертикальная черта означает или `(\+7|8)` Соответственно будет найдено +7 или 8.

После этого у нас уже пойдут одинаковые блоки, отличающиеся только количеством цифр `.*?(\d{3})` Точка ищет любой символ кроме переноса строки и звёздочка определяет количество вхождений от 0 и выше. Поэтому если в телефоне между +7/8 и следующим трёхзначным числом будут весьма часто встречающиеся скобки или пробелы, то телефон будет захвачен.

Большую роль здесь играет метасимвол `?` который делает наше жадное выражение `.*` ленивым. То есть поиск идёт до 1-го вхождения. Пример жадного и ленивого поиска:

```
import re

text = 'Бонд, Джеймс Бонд!'
res1 = re.findall(r'(.*)Бонд', text)
res2 = re.findall(r'(.*)Бонд', text)
print(''.join(res1))
print(''.join(res2))
```

```
res x
Бонд, Джеймс
, Джеймс
```

То есть жадный поиск забирает всё до последнего вхождения нашего шаблона, тогда как ленивый даст чистый результат, без вхождений шаблона, когда их более одного. А здесь пример наоборот – в группировке текст, и ленивый вариант забирает все вхождения текста.

```
import re

text = 'Бонд, Джеймс Бонд!'
res1 = re.findall(r'.*(Бонд)', text)
res2 = re.findall(r'.*?(Бонд)', text)
print(''.join(res1))
print(''.join(res2))
```

```
res x
Бонд
БондБонд
```

А вот и наша регулярка для поиска телефонов.

```
import re

s = 'Мой домашний телефон 8(812)177-66-56 и мобильные\
+7(921)8916235, 8 911 2517878, +7(952)348-54-85'
tel = re.findall(r'(\+7|8).*?(\d{3}).*?(\d{3}).*?(\d{2}).*?(\d{2})', s)
for i in tel:
    print(''.join(i))
```

```
reg x
88121776656
+79218916235
89112517878
+79523485485
```

Как видно на выходе, мы в таком случае получим номера, очищенные от всяких лишних символов. А так как мы получаем список кортежей, то для преобразования в строки и вывода столбиком, применяем цикл с методом `join`.